



INF1022

Trabalho Final - Analisadores Léxicos e Sintáticos

TRANSPILADOR OBSACT

2410804 Guilherme Machado de Bragança Soares

2410531 Jean Michel Jauffret de Oliveira

Professores:

Vitor Pinheiro de Almeida
Luis Henrique Costa Neto

Monitor:

Gabriel Moreira de
Cantuarina Farias

Link do Repositório (GitHub): <https://github.com/GuilhermeMBS/Obsact-Transpiler>

Rio de Janeiro
Junho 2026

RESUMO

Este trabalho apresenta o desenvolvimento de um transpilador completo para a linguagem de automação residencial e IoT denominada ObsAct. O sistema foi construído utilizando as ferramentas clássicas de engenharia de compiladores Flex (Analisador Léxico) e Bison (Analisador Sintático/Semântico), mapeando o código fonte original diretamente para a linguagem nativa C.

O projeto foi empacotado em uma estrutura de arquivos limpa e automatizada, na qual todo o código-fonte do transpilador, os módulos runtime de suporte e as suítes completas de testes estão organizados em um único diretório, compactado e anexado a este relatório no arquivo *projeto.zip*.

SUMÁRIO

1. IMPLEMENTAÇÕES	6
1.1. Analisador Léxico	6
1.2. Analisador Sintático e Semântico	6
1.3. Ambiente de Runtime Decoupled	7
2. GRAMÁTICA UTILIZADA	8
2.1. Símbolos Não-Terminais da Linguagem	8
2.2. Regras de Produção da Gramática Final	8
2.3. Justificativa das Regras Adicionadas ou Modificadas	10
2.3.1. Fatoração WITH_OBS	10
2.3.2. Encadeamento Recursivo em OBS	10
2.3.3. Enquadramento Sólido do Ponto Final (.)	10
3. FUNCIONALIDADES ADICIONAIS	11
3.1. Mecanismo de Alerta em Massa (Broadcast)	11
3.2. Rastreamento de Estado Persistente de Hardware	11
4. FUNCIONAMENTOS	12
4.1. O que funciona	12
4.2. O que não funciona	12
5. CASOS DE TESTE	13
6. INSTRUÇÕES DE EXECUÇÃO	14
6.1. Compilação do Transpilador	14
6.2. Execução da Suíte Completa de Transpilação	14
6.3. Execução dos Arquivos Gerados em C	15
6.4. Depuração Rápida de Arquivo Individual	15
6.5. Execução de um Arquivo C Intermediário Isolado	15

1. IMPLEMENTAÇÕES

O transpilador foi projetado seguindo uma arquitetura modular moderna e o princípio de Separação de Preocupações (*Separation of Concerns*). O pipeline de compilação está dividido nos seguintes módulos: analisador léxico, analisador sintático e ambiente *de runtime decoupled*.

1.1. Analisador Léxico (`src/lexical/lexical.l`)

Responsável pela varredura por casamento maximal de texto e conversão do fluxo de caracteres em tokens válidos.

Diferente de uma abordagem unificada rudimentar, a palavra-chave dispositivo e o caractere de dois-pontos (:) foram separados no léxico. Isso garante que o transpilador aceite espaços arbitrários entre eles, tornando o compilador robusto contra falhas de digitação e formatação.

Há também a implementação direta de checagem de tamanho limite para literais de mensagem (*msg*) e identificadores gerais (*TOKEN_IDENTIFIER*) utilizando a variável interna *yy leng* do Flex. Caso qualquer string correspondente a esses tokens ultrapasse 100 caracteres, o compilador emite um erro léxico explícito no *stderr* e aborta imediatamente a execução, blindando nativamente o tamanho de mensagens, nomes de dispositivos e de sensores.

1.2. Analisador Sintático e Semântico (`src/syntax/syntax.y`)

Implementa a gramática livre de contexto (GLC) e as ações semânticas que geram a árvore de sintaxe abstrata traduzida para C.

Validação Semântica de Contexto: Como o Flex opera sem contexto, a restrição de que o nome de um dispositivo (*namedevice*) pode conter apenas letras foi delegada ao Bison. Ao reduzir a regra sintática, o interpretador percorre os caracteres da string através da função *isalpha()*. Se houver dígitos numéricos (ex: *Ventilador1*), um erro semântico é disparado no console e o processo é interrompido via macro *YYABORT*, impedindo a geração do código C corrompido.

Relatório de Erros Robusto: Ativação da diretiva *%define parse.error detailed*, substituindo a mensagem genérica de erro por um log analítico que indica

exatamente qual token inesperado causou a falha e quais eram esperados pelo parser.

1.3. Ambiente de Runtime Decoupled (C nativo)

Em vez de injetar toda a lógica de persistência e gerenciamento dentro do código transpilado, a infraestrutura foi isolada em uma biblioteca interna dividida em dois subcomponentes em *src/functions/*.

aux_func.c (Gerenciador de Estado Privado): Aloca uma tabela de símbolos estática em memória para monitorar o estado binário interno (ON/OFF) de até 50 dispositivos concorrentes.

obsact_func.c (API de Comando Pública): Implementa as assinaturas requisitadas pelo laboratório (ligar, desligar, verificar), garantindo o comportamento e a formatação exata dos prints exigidos.

2. GRAMÁTICA UTILIZADA

Abaixo está listada a gramática formal livre de contexto consolidada e utilizada pelo motor do Bison. A notação foi adaptada para manter estrita similaridade com a descrição inicial apresentada no documento do laboratório, incluindo a introdução do símbolo ϵ (épsilon) para representar transições opcionais vazias.

2.1. Símbolos Não-Terminais da Linguagem

As produções da linguagem envolvem os seguintes símbolos não-terminais: *PROGRAM*, *DEVICES*, *DEVICE*, *WITH_OBS*, *CMDS*, *CMD*, *ATTRIB*, *EXP*, *VALUE*, *OBSACT*, *OBS*, *ACT*, *ACTEXECUTE*, *ACTALERT*, *DEVICE_LIST* e *ACTION*.

2.2. Regras de Produção da Gramática Final

Símbolo Não-Terminal	Regras de Produção
PROGRAM	→ DEVICES CMDS DEVICES CMDS .
DEVICES	→ DEVICES DEVICE DEVICE
DEVICE	→ dispositivo: { namedevice WITH_OBS }
WITH_OBS	→ , observation ϵ
CMDS	→ CMDS CMD CMD
CMD	→ ATTRIB . ACT . OBSACT
ATTRIB	→ set observation = EXP
EXP	→ VALUE ACTEXECUTE
VALUE	→ num bool
OBSACT	→ se OBS entao CMD se OBS entao CMD senao CMD
OBS	→ observation oplogic VALUE OBS && OBS
ACT	→ ACTEXECUTE ACTALERT
ACTEXECUTE	→ ACTION namedevice

ACTALERT	→ enviar alerta (msg WITH_OBS) namedevice enviar alerta (msg WITH_OBS) para todos: DEVICE_LIST
DEVICE_LIST	→ namedevice DEVICE_LIST , namedevice
ACTION	→ ligar desligar verificar

2.3. Justificativa das Regras Adicionadas ou Modificadas

2.3.1. Fatoração WITH_OBS

Criada como uma regra opcional (*épsilon*) para evitar ambiguidade posicional. Ela é reaproveitada tanto no registro de sensores atrelados aos dispositivos quanto na injeção de variáveis numéricas em strings de alerta.

2.3.2. Encadeamento Recursivo em OBS

A produção original foi expandida para suportar *OBS && OBS*, permitindo que expressões relacionais sejam conectadas infinitamente através de conjunções lógicas para avaliações de cenários IoT complexos.

2.3.3. Enquadramento Sólido do Ponto Final (.)

A regra CMD foi cuidadosamente projetada para exigir o ponto final como caractere terminador obrigatório em atribuições (*ATTRIB .*) e ações simples (*ACT .*). No entanto, ela remove essa exigência para blocos condicionais (*OBSACT*), visto que a instrução interna executada no escopo do *entao* ou *senao* já carrega consigo o ponto final de encerramento.

3. FUNCIONALIDADES ADICIONAIS

Além de cobrir todos os requisitos mínimos descritos no roteiro, a dupla projetou e codificou recursos extras voltados à usabilidade e completude do ambiente de automação:

3.1. Mecanismo de Alerta em Massa (Broadcast)

Adição da palavra-chave para todos: seguida por uma lista encadeada por vírgulas de tamanho dinâmico (*DEVICE_LIST*). O transpilador gera um algoritmo em C baseado em tokenização de strings em tempo de execução (*strtok*), distribuindo a mensagem de alerta simultaneamente para todos os alvos mapeados.

3.2. Rastreamento de Estado Persistente de Hardware

Em conformidade com o comportamento físico real, o utilitário de runtime monitora as invocações prévias de ligar e desligar no decorrer do programa de forma individual por dispositivo. Assim, chamadas subsequentes à função verificar retornam o estado lógico exato do componente, abandonando prints fixos ou estáticos simulados.

4. FUNCIONAMENTOS

4.1. O que funciona

O sistema apresenta mapeamento exato de inicialização com ou sem sensores, além de resolução de escopo para encadeamento lógico de cláusulas && (AND). Ocorre o abortamento imediato com deleção do arquivo corrompido em caso de estouro de tamanho de string com mais de 100 caracteres em mensagens ou identificadores, o bloqueio completo de uso de números em variáveis de dispositivo e a transpilação bem-sucedida dos arquivos válidos com o devido barramento dos inválidos presentes na suíte de testes padrão.

4.2. O que não funciona

Não foram identificadas falhas ou omissões no escopo planejado. Vale destacar que, por especificação de design da linguagem base, a estrutura condicional aninha apenas um único comando completo (*cmd*) por ramificação, impossibilitando a execução de múltiplos comandos em sequência dentro de um único bloco *se...entao* sem a declaração explícita de novos condicionais.

5. CASOS DE TESTE

Diretório	Descrição
tests/examples/	Esta pasta contém os 10 arquivos <i>.obsact</i> fornecidos originalmente no documento do laboratório, abrangendo do <i>test1.obsact</i> ao <i>test10.obsact</i> .
tests/our_tests/	Esta pasta reúne arquivos customizados pela dupla, incluindo o <i>test_max_chars.obsact</i> para validar estouro de caracteres no Flex, o <i>test_invalid_device.obsact</i> para testar a validação semântica de identificadores no Bison, e o <i>test_complex_logic.obsact</i> para verificar a pontuação e expressões lógicas complexas.

Todos os scripts de teste e validação foram empacotados e estão disponíveis na raiz do diretório do projeto, segmentados em duas pastas de controle para preservar a integridade das avaliações.

6. INSTRUÇÕES DE EXECUÇÃO

O ambiente de automação foi desenvolvido sobre o utilitário GNU Make e otimizado para executar nativamente em terminais Windows (via PowerShell ou Prompt de Comando) que possuam o GCC e as portas de binários do Flex/Bison instaladas. Para garantir isolamento absoluto e logs completamente limpos, os comandos foram configurados no modo silencioso (@), ocultando códigos internos de shell e avisos irrelevantes do compilador.

6.1. Compilação do Transpilador

Para gerar os analisadores estruturais e consolidar o binário executável principal (transpiler.exe), utiliza-se o comando: *mingw32-make all*

6.2. Execução da Suíte Completa de Transpilação

Para ler todos os arquivos de teste das duas pastas, realizar a validação semântica/léxica e gerar os respectivos arquivos de saída em C estruturados dentro da pasta interna out/, executa-se o comando: *mingw32-make tests*.

Arquivos que contêm erros de sintaxe propositais da suíte do PDF exibirão o log analítico de erro de linha na tela e interromperão a escrita do C para aquele arquivo específico, mantendo o diretório limpo.

6.3. Execução dos Arquivos Gerados em C

Para compilar em lote os códigos C gerados integrando-os à biblioteca runtime modular e executar os resultados na tela de forma sequencial, utiliza-se a instrução: *mingw32-make run-tests*.

6.4. Depuração Rápida de Arquivo Individual

Depuração Rápida de Arquivo Individual Para transpilar, compilar nativamente e rodar um arquivo de código-fonte individual de forma imediata em uma única linha de comando, adota-se a seguinte linha:

mingw32-make test-single FILE=tests/our_tests/test_complex_logic.obsact

6.5. Execução de um Arquivo C Intermediário Isolado

Para compilar e rodar individualmente um arquivo .c que já foi gerado na pasta out, o que se mostra útil para validar de forma isolada o comportamento do gerenciador de estados, executa-se o comando:

```
mingw32-make run-single FILE=out/our_tests/test_complex_logic.c
```